Alla ricerca di Peach

Rappresentazione, Ricerca e Ragionamento in Prolog

Autore: Cannone Giuseppe
Matricola: 774773
Email: g.cannone30@studenti.uniba.it
GitHub: `https://github.com/pinopostino/SuperMario-`

Anno Accademico 2024/2025

Indice

1 Introduzione **2**

2 Rappresentazione della Conoscenza **2**
2.1 Fatti della Base di Conoscenza 3
2.2 Regole della Base di Conoscenza 4
2.3 Vantaggi e Limiti della Rappresentazione 5

3 Ricerca nello Spazio degli Stati **5**
3.1 Modellazione come Spazio di Stati 6
3.2 Algoritmi Implementati 6
3.3 Struttura dell'Implementazione 6
3.4 Valutazione Sperimentale 7
3.5 Discussione dei Risultati 7
3.6 Conclusioni e Estendibilità 7

4 Ragionamento Automatico **7**
4.1 Inferenza sui Costi e sugli Ostacoli 7
4.2 Verifica della Validità delle Azioni 8
4.3 Risoluzione del Labirinto con Ragionamento Logico 8
4.4 Integrazione Python-Prolog con Interfaccia Interattiva 8
4.5 Modularità del Ragionamento 9
4.6 Conclusioni 9

5 Scelte Progettuali, Interfaccia Utente e Conclusioni **9**
5.1 Motivazioni delle Scelte Progettuali 9
5.2 Integrazione con Interfaccia Grafica 10
5.3 Spunti di Estensione 10
5.4 Visualizzazione dell'Interfaccia Grafica (GUI) 10
5.5 Conclusioni Finali 14

1. Introduzione

Il progetto si propone di sviluppare un sistema **basato su conoscenza** per la risoluzione automatica di un **labirinto 12x12 di Bowser** ambientato nel mondo di Super Mario, con l'obiettivo finale di trovare il **percorso ottimale** per salvare la Principessa Peach. L'applicazione integra metodologie di *rappresentazione della conoscenza*, *algoritmi di ricerca nello spazio degli stati*, *logica dichiarativa* e *interazione utente* tramite un'interfaccia grafica sviluppata in **Python**.

La mappa del mondo viene rappresentata in **Prolog** attraverso una *base di conoscenza* che descrive lo stato dell'ambiente, includendo nemici, ostacoli e personaggi. Ogni cella della mappa è modellata come un nodo in un grafo, mentre i movimenti ammessi costituiscono gli archi. Le regole logiche codificano la validità delle azioni e i costi di attraversamento associati a ciascun tipo di ostacolo. Il sistema implementa diverse strategie di ricerca (*DFS*, *A**, *Ricerca Bidirezionale*) per esplorare lo spazio delle soluzioni e determinare il percorso più efficiente in termini di **costo e lunghezza**.

L'interfaccia grafica, sviluppata con la libreria `tkinter`, consente all'utente di interagire con il sistema selezionando l'algoritmo di risoluzione desiderato, visualizzando il labirinto in tempo reale e osservando l'animazione del percorso trovato. L'output include informazioni analitiche come il **numero di mosse**, il **costo complessivo** e il **tempo di esecuzione**.

In relazione agli argomenti del corso, il progetto affronta i seguenti temi chiave:

- **Ricerca di soluzioni in spazi di stati**: il problema è modellato come un grafo di navigazione, esplorato mediante algoritmi di ricerca *informata e non informata*. L'ambiente è generato dinamicamente e la validità delle soluzioni è garantita tramite *verifica euristica*.

- **Rappresentazione e ragionamento proposizionale e relazionale**: la descrizione del mondo avviene attraverso *fatti* e *regole* in Prolog (clausole di Horn), utilizzate per codificare la dinamica dell'ambiente e le *relazioni spaziali* tra gli elementi della mappa.

- **Ragionamento automatico**: il motore logico viene sfruttato per *inferire la validità dei percorsi*, *calcolare costi cumulativi* e determinare il superamento di ostacoli sulla base delle *regole dichiarative*.

Il progetto dimostra come sia possibile applicare in maniera sinergica diverse tecniche dell'**Ingegneria della Conoscenza** per risolvere un problema pratico e ben definito, evidenziando anche il potenziale di estensione del sistema con componenti di *apprendimento automatico* o moduli di *ragionamento probabilistico*.

2. Rappresentazione della Conoscenza

In questo progetto, la rappresentazione della conoscenza gioca un ruolo cruciale nella modellazione del mondo di gioco ispirato all'universo di Super Mario. Il sistema è stato concepito per descrivere formalmente l'ambiente in cui Mario si muove alla ricerca della Principessa Peach, attraverso una struttura logica chiara e manipolabile.

La mappa 12x12 viene considerata come un grafo i cui nodi rappresentano le celle della griglia, mentre gli archi corrispondono ai movimenti consentiti tra le celle adiacenti. Ogni cella può contenere elementi diversi: ostacoli invalicabili (come muri), ostacoli

attraversabili con un certo costo (come la lava o i nemici), oppure essere una semplice casella libera. La conoscenza di questo ambiente viene espressa mediante fatti e regole logiche in linguaggio Prolog, rendendo possibile l'integrazione con algoritmi di ricerca per determinare percorsi ottimali.

La scelta di Prolog come linguaggio di rappresentazione è motivata dalla sua natura dichiarativa e dalla sua capacità di modellare problemi complessi tramite inferenza logica. Tutte le informazioni rilevanti vengono codificate sotto forma di predicati logici, consentendo al sistema di rispondere a interrogazioni, generare dinamicamente mappe coerenti e supportare strategie di esplorazione intelligenti.

2.1. Fatti della Base di Conoscenza

Nel sistema proposto, la conoscenza sul mondo di gioco è rappresentata attraverso **fatti dinamici**. I fatti sono entità atomiche, espresse tramite predicati, che modellano gli oggetti presenti nella mappa, come personaggi, ostacoli e nemici, e ne descrivono le caratteristiche statiche o dinamiche.

I fatti costituiscono la componente più semplice della base di conoscenza e definiscono l'ambiente su cui gli algoritmi di ricerca possono operare. In particolare, ogni fatto rappresenta una singola informazione verificabile sullo stato del labirinto. Poiché la mappa viene rigenerata ad ogni esecuzione, è necessario che i fatti siano dichiarati come `dynamic`, cioè modificabili a runtime.

I principali fatti utilizzati nel sistema includono:

- `dimensione/2`: definisce le dimensioni della griglia di gioco, tipicamente 12 righe per 12 colonne.
- `mario/2`: rappresenta la posizione iniziale del protagonista Mario nella mappa.
- `peach/2`: indica la posizione finale da raggiungere, ovvero dove si trova Peach.
- `muro/2`: individua le celle contenenti muri, che fungono da ostacoli invalicabili.
- `lava/2`: rappresenta celle pericolose attraversabili con costo elevato.
- `goomba/2`, `spiny/2`, `piranha/2`, `boss/2`: identificano la presenza di nemici. Sebbene siano attraversabili, comportano un costo maggiore.
- `percorso_memorizzato/1`: utilizzato per memorizzare il percorso trovato da un algoritmo di ricerca.
- `get_mappa/1`: restituisce la rappresentazione simbolica della mappa per la GUI.

Questi fatti vengono generati dinamicamente dal sistema ogni volta che si richiama il predicato `genera_mappa/0`, il quale crea una configurazione del labirinto.

Esempio di definizione dei fatti:

```
mario(11, 0).
peach(0, 11).
muro(4, 5).
lava(2, 3).
goomba(6, 7).
piranha(5, 2).
boss(3, 9).
```

Grazie a questa modellazione, l'intero ambiente di gioco può essere descritto in maniera modulare e interrogabile, facilitando la progettazione di regole e algoritmi.

2.2. Regole della Base di Conoscenza

Le **regole** modellano il comportamento del sistema, permettono di dedurre nuove informazioni a partire dai fatti e guidano l'esecuzione degli algoritmi di ricerca.

Regole di Adiacenza

Queste regole definiscono i movimenti possibili tra celle adiacenti. Mario può spostarsi in alto, in basso, a destra o a sinistra, purché rimanga all'interno dei limiti della mappa:

```
adiacente([X,Y], [X1,Y]) :- X1 is X - 1, X1 >= 0.     %
    Su
adiacente([X,Y], [X1,Y]) :- X1 is X + 1, X1 < 12.     %
    Gi
adiacente([X,Y], [X,Y1]) :- Y1 is Y - 1, Y1 >= 0.     %
    Sinistra
adiacente([X,Y], [X,Y1]) :- Y1 is Y + 1, Y1 < 12.     %
    Destra
```

L'ordine delle regole non è significativo nel comportamento della ricerca, poiché le strategie di visita (DFS, A*, BD) determinano la priorità di esplorazione.

Validità delle celle

Questa regola verifica se una cella può essere attraversata. In particolare, una cella è considerata valida se non è un muro:

```
casella_valida_semplice([X,Y]) :-
    X >= 0, X < 12,
    Y >= 0, Y < 12,
    \+ muro(X,Y).
```

Calcolo del costo di attraversamento

Ogni cella ha un costo diverso in base al tipo di ostacolo che contiene. Mario può "saltare" i nemici ma ciò richiede più tempo, simulato attraverso un incremento del costo. Le celle vuote hanno costo 1:

```
calcola_costo_semplice([X,Y], Costo) :-
    (boss(X,Y) -> Costo = 8 ;
     lava(X,Y) -> Costo = 6 ;
     piranha(X,Y) -> Costo = 5 ;
     spiny(X,Y) -> Costo = 4 ;
     goomba(X,Y) -> Costo = 2 ;
     Costo = 1).
```

Distribuzione dinamica dei nemici

Queste regole posizionano nemici e ostacoli in celle casuali :

```
piazza_nemico([R,C], goomba)  :- asserta(goomba(R, C)).
piazza_nemico([R,C], spiny)   :- asserta(spiny(R, C)).
piazza_nemico([R,C], piranha) :- asserta(piranha(R, C))
    .
piazza_nemico([R,C], boss)    :- asserta(boss(R, C)).
piazza_nemico([R,C], lava)    :- asserta(lava(R, C)).
```

Queste regole sono invocate automaticamente da `genera_mappa/0` durante la costruzione dell'ambiente di gioco.

2.3. Vantaggi e Limiti della Rappresentazione

La scelta di utilizzare Prolog per modellare la conoscenza nel contesto del labirinto di Super Mario presenta diversi punti di forza, ma anche alcune criticità che meritano attenzione.

Vantaggi

- **Dichiaratività e chiarezza**: la rappresentazione logica mediante fatti e regole consente una modellazione intuitiva e leggibile dello stato del mondo, favorendo la manutenzione e l'estensione del sistema.
- **Separazione tra conoscenza e comportamento**: la logica di rappresentazione è separata dagli algoritmi di ricerca, favorendo la modularità e il riuso del codice.
- **Espressività inferenziale**: le regole logiche consentono di derivare in modo automatico condizioni complesse, come la validità delle celle o il costo complessivo del percorso.

Limiti

- **Inferenza deterministica**: il motore inferenziale non supporta incertezza, priorità tra regole o ragionamento probabilistico, limitando scenari più realistici o complessi.
- **Scalabilità**: l'aumento delle dimensioni della mappa o della complessità logica può portare a una degradazione delle performance o a un aumento significativo della complessità computazionale.
- **Dipendenza dal dominio**: il sistema è fortemente legato al dominio specifico (Super Mario), e richiede modifiche non banali per essere adattato ad altri contesti.

3. Ricerca nello Spazio degli Stati

La fase di ricerca costituisce il nucleo computazionale del sistema, in quanto consente di determinare il percorso che Mario deve intraprendere per salvare la Principessa Peach in un ambiente ostile e dinamico. La mappa viene modellata come una griglia 12x12, popolata da nemici e ostacoli, e ogni cella può essere considerata uno stato del sistema. L'obiettivo della ricerca è individuare una sequenza di azioni che consenta di passare dallo stato iniziale (la posizione di Mario) allo stato obiettivo (la posizione di Peach), minimizzando il costo complessivo del percorso.

3.1. Modellazione come Spazio di Stati

La mappa viene rappresentata implicitamente come un grafo orientato, in cui:

- ogni nodo corrisponde a una cella della griglia;
- gli archi rappresentano le possibili transizioni tra celle adiacenti (movimenti nelle quattro direzioni cardinali);
- ogni transizione ha un peso, che rappresenta il costo di attraversamento della cella di destinazione, in funzione del tipo di contenuto: Goomba (2), Lava (6), Boss (8), cella vuota (1), ecc.

In termini funzionali, il problema è modellato come segue:

- **Stato iniziale**: posizione di Mario (es. (11, 0));
- **Stato obiettivo**: posizione della Principessa Peach (es. (0, 11));
- **Azioni**: movimento verso celle adiacenti non invalicabili;
- **Funzione di transizione**: restituisce la nuova posizione dopo l'azione;
- **Funzione di costo**: restituisce un intero positivo associato alla cella di destinazione;
- **Euristica** (solo per A*): distanza di Manhattan tra lo stato corrente e l'obiettivo.

3.2. Algoritmi Implementati

Nel progetto sono stati implementati tre diversi algoritmi di ricerca, ognuno caratterizzato da proprietà e strategie differenti:

- **DFS (Depth-First Search)**: algoritmo non informato che esplora in profondità ogni ramo prima di tornare indietro. Sebbene semplice, tende a generare percorsi lunghi e subottimali.
- **A***: algoritmo informato che combina il costo del percorso già compiuto (g) con una funzione euristica (h), ovvero la distanza di Manhattan. Garantisce soluzioni ottimali con un numero contenuto di espansioni.
- **Ricerca Bidirezionale**: due ricerche simultanee (una da Mario e una da Peach) che cercano un punto di incontro. Riduce lo spazio esplorato, ma può generare molte inferenze.

3.3. Struttura dell'Implementazione

Ogni algoritmo è stato definito in un modulo dedicato in linguaggio Prolog. Il sistema richiama i predicati `risolvi_dfs/3`, `risolvi_astar/3` e `risolvi_bd/3`. Tutti i percorsi si basano sulla configurazione generata dinamicamente da `genera_mappa/0`.

Le mosse ammissibili sono regolate da `adiacente/2` e `casella_valida_semplice/1`, mentre i costi di movimento sono calcolati tramite `calcola_costo_semplice/2`.

In questa valutazione, non è stato riportato il tempo di computazione in quanto influenzato da fattori esterni al motore logico (es. chiamate Python-Prolog, rendering grafico).

3.4. Valutazione Sperimentale

La valutazione è stata condotta su **50 esecuzioni randomiche**(il numero di eseccuzioni può variare), utilizzando un wrapper in Python via PySwip, il quale esegue il predicato `confronta_algoritmi/0` per ciascuna prova. Le metriche raccolte sono:

- **Lunghezza**: numero di celle del percorso trovato;
- **Costo Totale**: somma dei pesi delle celle attraversate.

I risultati medi finali sono riportati nella tabella seguente:

Algoritmo	Lunghezza Media	Costo Medio
A*	24.85	31.14
DFS	64.20	91.95
Ricerca Bidirezionale	23.00	30.95

3.5. Discussione dei Risultati

I risultati mostrano che **Ricerca Bidirezionale** ottiene la miglior lunghezza media del percorso (23.0 celle) e il miglior costo medio (30.95), sebbene spesso comporti un carico computazionale maggiore. L'**A*** si dimostra molto competitivo, offrendo un buon compromesso tra costo e prestazioni. Al contrario, **DFS** risulta inefficiente sia in termini di lunghezza che di costo, confermando i limiti delle strategie non informate in ambienti complessi.

3.6. Conclusioni e Estendibilità

La combinazione di strategie di ricerca permette di adattare la risoluzione al contesto della mappa. L'architettura modulare in Prolog favorisce inoltre l'integrazione futura di algoritmi più sofisticati (es. Beam Search, Iterative Deepening A*) o di tecniche di apprendimento per la generazione dinamica di euristiche, estendendo così le capacità decisionali del sistema.

4. Ragionamento Automatico

Il motore di inferenza logica implementato in Prolog costituisce il nucleo del ragionamento automatico nel progetto *Alla Ricerca di Peach*. Grazie alla natura dichiarativa del linguaggio e all'utilizzo di predicati dinamici, il sistema è in grado di adattarsi dinamicamente alla configurazione della mappa, verificare la validità delle azioni, calcolare costi e risolvere autonomamente istanze complesse del problema.

4.1. Inferenza sui Costi e sugli Ostacoli

Il comportamento degli agenti nel mondo (Mario, Peach, nemici, ostacoli) è governato da un insieme di regole logiche. Un esempio centrale è il predicato:

```
calcola_costo_semplice([X,Y], Costo) :-
    (boss(X,Y) -> Costo = 8
    ; lava(X,Y) -> Costo = 6
    ; piranha(X,Y) -> Costo = 5
```

```
        ; spiny(X,Y) -> Costo = 4
        ; goomba(X,Y) -> Costo = 2
        ; Costo = 1).
```

Questa regola permette al sistema di dedurre il costo di attraversamento di ogni cella, a seconda del tipo di entità presente. L'inferenza è automatica, evitando controlli imperativi e consentendo un calcolo cumulativo del percorso direttamente durante la navigazione della mappa.

4.2. Verifica della Validità delle Azioni

Il sistema valuta la validità di ogni azione attraverso il predicato `casella_valida_semplice/1`:

```
casella_valida_semplice([X,Y]) :-
    X >= 0, X < 12, Y >= 0, Y < 12,
    \+ muro(X,Y).
```

Associato al predicato `adiacente/2`, che definisce le mosse cardinali possibili, esso permette di inferire se una cella è accessibile da un'altra. In questo modo, ogni transizione da uno stato all'altro avviene solo se logicamente ammissibile, rispettando i vincoli spaziali e semantici della mappa.

4.3. Risoluzione del Labirinto con Ragionamento Logico

Il progetto include tre predicati logici che implementano algoritmi di ricerca intelligenti, ognuno dei quali sfrutta il ragionamento dichiarativo per navigare nel labirinto:

- `risolvi_dfs/3`: ricerca in profondità, utilizza un approccio esplorativo deterministico;
- `risolvi_astar_semplice/3`: implementa A*, combinando costo effettivo e stima euristica (distanza di Manhattan);
- `risolvi_bd/3`: esegue ricerca bidirezionale da Mario e da Peach, trovando un punto di incontro.

Ciascuno di questi predicati viene eseguito dinamicamente sulla mappa generata, e produce un percorso risolutivo che rappresenta la sequenza di azioni necessarie per completare il gioco. L'output è una lista di coordinate che costituisce la soluzione logica al problema.

4.4. Integrazione Python-Prolog con Interfaccia Interattiva

Il motore logico Prolog è integrato all'interno di una GUI interattiva realizzata in Python (Tkinter). Tale integrazione è resa possibile grazie alla libreria `PySwip`, che permette l'esecuzione diretta di query Prolog da codice Python.

Ogni algoritmo di ricerca è incapsulato in un predicato wrapper come `mario_astar_gui(Output)`, che redireziona l'output logico in una stringa grazie a:

```
capture_output(Goal, Output) :-
    with_output_to(atom(Output), Goal).
```

Questa stringa viene successivamente analizzata da Python, che estrapola la mappa ASCII e i dati di performance, li parsifica, e li visualizza graficamente nella GUI come una griglia animata, con colori e simboli coerenti (Mario, Peach, nemici, ostacoli, percorso AI).

L'interazione è completamente bidirezionale: l'utente può scegliere l'algoritmo, attivare la risoluzione e osservare il comportamento in tempo reale, con log dettagliato, animazione del percorso e statistiche finali.

4.5. Modularità del Ragionamento

Tutti i predicati logici del motore sono dichiarati dinamici:

```
:- dynamic mario/2, peach/2, muro/2, lava/2, goomba/2,
   spiny/2, piranha/2, boss/2.
```

Grazie all'utilizzo di `asserta/1` e `retractall/1`, la base di conoscenza può essere aggiornata in tempo reale ad ogni nuova generazione della mappa. Questo garantisce:

- la reattività del sistema alle modifiche ambientali;
- la validazione automatica dei percorsi generati;
- la possibilità futura di gestire condizioni avanzate (power-up, cambi dinamici del mondo).

4.6. Conclusioni

Il ragionamento automatico, implementato attraverso logica dichiarativa in Prolog, fornisce al sistema una capacità inferenziale trasparente, adattiva ed espandibile. Ogni cella attraversata, ogni scelta computazionale di Mario, ogni percorso calcolato, sono il risultato di inferenze logiche valide, tracciabili e spiegabili.

L'integrazione con Python consente di tradurre questo ragionamento in un'esperienza utente interattiva ,dimostrando l'efficacia del ragionamento automatico in un contesto ludico ma computazionalmente complesso.

5. Scelte Progettuali, Interfaccia Utente e Conclusioni

5.1. Motivazioni delle Scelte Progettuali

Il progetto *Alla ricerca di Peach* è stato concepito come un ambiente ludico ma rigoroso, in grado di integrare i principali paradigmi dell'Intelligenza Artificiale simbolica. La scelta di rappresentare la mappa come una griglia 12x12 riflette la necessità di un dominio sufficientemente complesso da permettere esplorazioni interessanti, ma contenuto al punto da garantire prestazioni accettabili durante l'esecuzione.

L'utilizzo del linguaggio Prolog per modellare la conoscenza e il processo decisionale è stato dettato dalla sua natura dichiarativa e dalla facilità con cui consente di definire regole, relazioni tra entità e comportamenti inferenziali. Prolog si è dimostrato ideale per:

- rappresentare lo stato del mondo tramite fatti dinamici;
- modellare relazioni spaziali;

- eseguire inferenze sui costi, sulla validità delle azioni e sulla transitività delle mosse;
- costruire e verificare percorsi ottimali in modo logico e dichiarativo.

La scelta di implementare algoritmi di ricerca classici come DFS, A* e Ricerca Bidirezionale consente di mettere a confronto approcci non informati, euristici e bidirezionali, dando spazio a considerazioni sia teoriche che pratiche sull'efficienza computazionale e sulla qualità delle soluzioni ottenute.

5.2. Integrazione con Interfaccia Grafica

Sebbene il cuore del progetto sia il motore logico in Prolog, si è scelto di affiancare una GUI in Python (`Tkinter`) per scopi didattici e dimostrativi. L'interfaccia, integrata tramite `PySwip`, consente all'utente di interagire con il sistema selezionando l'algoritmo desiderato, avviando l'esplorazione e osservando visivamente il risultato sotto forma di mappa animata.

Questa integrazione è utile per:

- migliorare la comprensione delle dinamiche dell'algoritmo;
- fornire feedback immediato e visivo sulla correttezza della soluzione;
- rendere più accessibile e coinvolgente l'interazione con il sistema.

Tuttavia, la GUI non costituisce il nucleo centrale del progetto, che resta focalizzato sull'aspetto simbolico e inferenziale della rappresentazione della conoscenza.

5.3. Spunti di Estensione

L'architettura modulare del progetto e l'impiego di predicati dinamici rendono semplice l'estensione del sistema. Alcuni possibili sviluppi futuri includono:

- **Integrazione di nuovi algoritmi di ricerca**, come Iterative Deepening A* o algoritmi genetici;
- **Introduzione di condizioni dinamiche**, come oggetti mobili o eventi temporizzati (es. muri che si aprono, nemici in movimento);
- **Uso di tecniche di apprendimento**, ad esempio reinforcement learning per far apprendere a Mario percorsi ottimali in scenari non noti;
- **Espansione della rappresentazione del mondo**, con nuove entità e regole di interazione (es. power-up che riducono il costo degli ostacoli);
- **Esperimenti su griglie più grandi**, per valutare la scalabilità degli algoritmi.

5.4. Visualizzazione dell'Interfaccia Grafica (GUI)

Il modulo `App.py` è responsabile della gestione dell'interfaccia grafica dell'applicazione *Super Mario – Alla ricerca di Peach*. La GUI è stata progettata per rendere l'esperienza di interazione intuitiva, accattivante e funzionale, soprattutto in ottica didattica. Di seguito viene descritta la sequenza delle schermate principali.

Panoramica visiva

1. Schermata di Benvenuto (Main Menu)

All'avvio dell'applicazione viene presentata una schermata semplice e stilisticamente coerente con l'universo di Super Mario. Il titolo principale e la sottoscritta *"A.I. Pathfinding Quest"* introducono il contesto. L'utente può:

- Premere **Play Game** per iniziare il gioco.
- Premere **Exit** per chiudere l'applicazione.

L'interfaccia richiama lo stile arcade retrò, con colori vivaci e font cartoon che migliorano l'ingaggio.

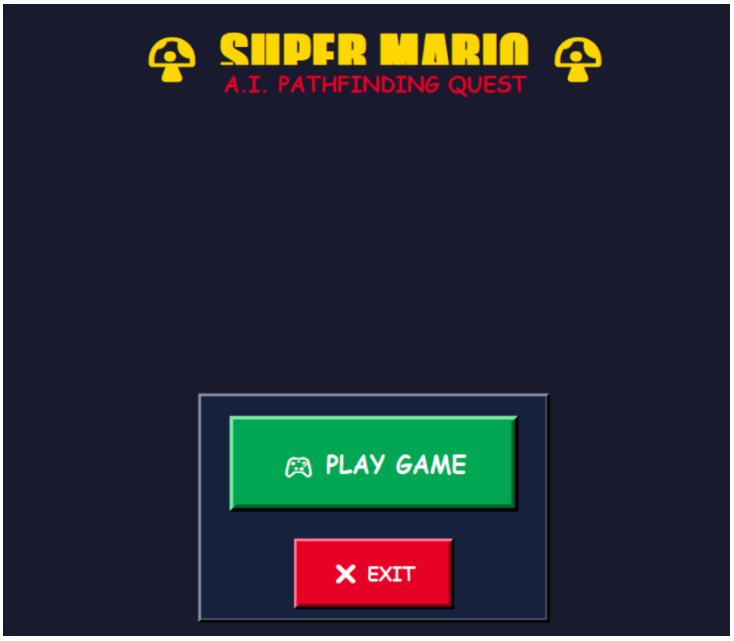

Figura 1: Schermata iniziale: Menu principale.

2. Introduzione alla Storia (Story Screen)

Una volta avviata la partita, viene narrata la storia: la Principessa Peach è stata rapita da Bowser, che ha costruito un labirinto pieno di insidie. Viene offerta una lista degli ostacoli presenti, come muri di mattoni, lava, Goomba, Spiny, Piante Piranha e boss. L'utente può scegliere se proseguire nella missione o tornare al menu principale.

Figura 2: Introduzione narrativa alla missione.

3. Selezione dell'Algoritmo (Algorithm Choice Screen)

Prima di iniziare la risoluzione del labirinto, il giocatore deve selezionare l'algoritmo di ricerca da utilizzare. Le opzioni disponibili sono:

- **Ricerca in Profondità (DFS)**: esplora i percorsi più profondi prima.
- **Ricerca Bidirezionale**: ricerca in parallelo da inizio e fine.
- **Algoritmo A***: sfrutta euristiche per bilanciare velocità e qualità.

Ogni scelta è accompagnata da una breve descrizione e un pulsante di selezione.

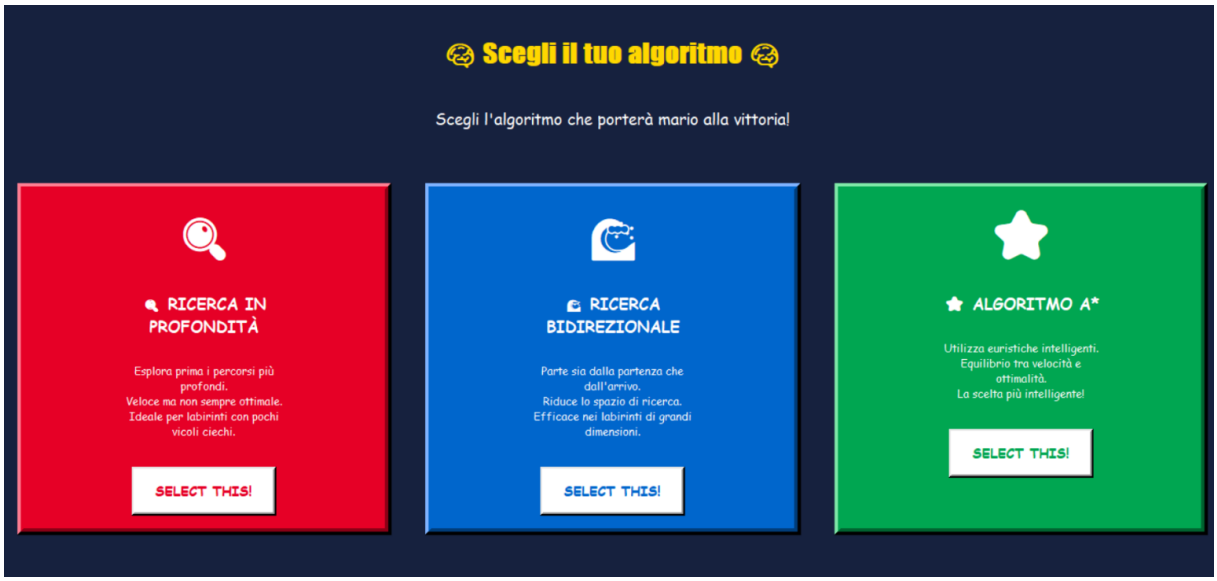

Figura 3: Selezione dell'algoritmo di ricerca.

4. Gameplay e Labirinto (Main Game Interface)

Questa schermata rappresenta il cuore interattivo dell'applicazione. Si visualizza il labirinto 12×12, con:

- Mario (posizione iniziale)
- Peach (obiettivo)
- Nemici e ostacoli vari
- Il percorso ottimale evidenziato con simboli

Sulla destra si trovano due pannelli:

- **Maze Legend**: legenda dei simboli utilizzati.
- **Mission Stats**: statistiche della risoluzione (tempo computazione AI, numero di passi, lunghezza del percorso, costo totale).

È presente anche un log in basso che fornisce feedback testuali in tempo reale sull'esecuzione dell'algoritmo selezionato.

Figura 4: Interfaccia di gioco con percorso ottimale calcolato.

Considerazioni Finali

La GUI realizzata in `App.py` non è un semplice abbellimento grafico, ma una componente progettata per:

- Semplificare l'interazione utente-algoritmo.
- Visualizzare con chiarezza le strategie di risoluzione.
- Rendere fruibili concetti complessi come il pathfinding e l'intelligenza artificiale.

Inoltre, il collegamento con il motore logico scritto in `Prolog` permette di sfruttare la potenza del ragionamento simbolico mantenendo un'interfaccia amichevole e narrativa.

5.5. Conclusioni Finali

Il progetto rappresenta un'applicazione significativa delle tecniche di Intelligenza Artificiale simbolica, dimostrando come sia possibile costruire un sistema autonomo in grado di ragionare, esplorare e prendere decisioni in un ambiente strutturato. L'integrazione tra logica dichiarativa, inferenza automatica e una semplice interfaccia grafica ha consentito di realizzare un prototipo funzionale, spiegabile e facilmente estendibile.

Attraverso il personaggio di Mario, si è potuto veicolare un modello computazionale coerente, che non si limita alla mera ricerca di percorsi, ma riflette capacità di ragionamento logico, valutazione comparativa e adattamento al contesto.

Il lavoro svolto getta le basi per ulteriori sperimentazioni e approfondimenti, offrendo spunti per lo studio di sistemi intelligenti applicati ad ambienti dinamici e complessi.